# 2025EM1200153_DDP_Assignment

## Preprocessing Workflow Pipeline

This section outlines the complete preprocessing workflow applied to the `fifa21 datatset.csv` dataset.

### Step 1: Raw Dataset Loaded

First, we mounted Google Drive and loaded the raw dataset into a pandas DataFrame. We then displayed the first few rows to get an initial look at the data.

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remoun
```

## Import Dateset fifa21 datatset.csv

```
import pandas as pd

# Adjust the path if your file is in a subdirectory within MyDrive
file_path = '/content/fifa21 datatset.csv'
df = pd.read_csv(file_path)

# Display the first 5 rows of the DataFrame
display(df.head())
```

```
/tmp/ipykernel_2739/3147445905.py:5: DtypeWarning: Columns (76) have m
  df = pd.read_csv(file_path)
```

| | ID | Name | LongName | photoUrl | |
|---|---|---|---|---|---|
| **0** | 158023 | L. Messi | Lionel Messi | https://cdn.sofifa.com/players/158/023/21_60.png | htt |
| **1** | 20801 | Cristiano Ronaldo | C. Ronaldo dos Santos Aveiro | https://cdn.sofifa.com/players/020/801/21_60.png | |
| **2** | 200389 | J. Oblak | Jan Oblak | https://cdn.sofifa.com/players/200/389/21_60.png | h |
| **3** | 192985 | K. De Bruyne | Kevin De Bruyne | https://cdn.sofifa.com/players/192/985/21_60.png | htt |
| **4** | 190871 | Neymar Jr | Neymar da Silva Santos Jr. | https://cdn.sofifa.com/players/190/871/21_60.png | http:/ |

5 rows × 77 columns

### Step 2: Data Exploration

We explored the dataset to understand its structure, identify the number of records and features, classify attribute types, and generate a data dictionary.

## Dataset Overview

```
# 1. Describe the dataset

# Number of records
num_records = len(df)
print(f"Number of records: {num_records}")

# Number of features
num_features = df.shape[1]
print(f"Number of features: {num_features}")

#A target variable would be the outcome we are trying to predict. In
print("Target variable: 'Overall', 'Value' and 'Potential'")
```

```
Number of records: 18979
Number of features: 77
Target variable: 'Overall', 'Value' and 'Potential'
```

# Attribute Type Classification

```
# 2. Identify attribute types

attribute_types = {}
for col in df.columns:
    dtype = str(df[col].dtype)
    nunique = df[col].nunique()
    ratio_unique = nunique / num_records

    if dtype == 'object':
        if nunique <= 50 and df[col].apply(lambda x: isinstance(x, st
            attribute_types[col] = 'Nominal'
        else:
            attribute_types[col] = 'Nominal/Text'
    elif 'int' in dtype or 'float' in dtype:
        if nunique == 2:
            attribute_types[col] = 'Numeric (Discrete/Binary)'
        elif 'int' in dtype and ratio_unique < 0.1:  # Heuristic for
            attribute_types[col] = 'Numeric (Discrete)'
        else:
            attribute_types[col] = 'Numeric (Continuous)'
    elif 'datetime' in dtype:
        attribute_types[col] = 'Temporal'
    else:
        attribute_types[col] = 'Other'

# These are often ratings or levels that imply order.
potential_ordinal_cols = [
    'Joined','POT', 'SM', 'A/W', 'D/W'
]

for col in potential_ordinal_cols:
    if col in attribute_types and ('Numeric' in attribute_types[col]
        # If it's numeric and already classified as discrete/continuo
        # If it's object, it's more likely nominal or ordinal if cate
        if 'Numeric' in attribute_types[col] and ratio_unique < 0.5:
             attribute_types[col] = 'Ordinal (Numeric)'
        elif attribute_types[col] == 'Nominal/Text': # If text based,
            attribute_types[col] = 'Ordinal (Categorical)'


print("Feature classification:")
for col, attr_type in attribute_types.items():
    print(f"- {col}: {attr_type}")
```

```
- Short Passing: Numeric (Discrete)
- Volleys: Numeric (Discrete)
- Skill: Numeric (Discrete)
- Dribbling: Numeric (Discrete)
- Curve: Numeric (Discrete)
- FK Accuracy: Numeric (Discrete)
- Long Passing: Numeric (Discrete)
- Ball Control: Numeric (Discrete)
- Movement: Numeric (Discrete)
- Acceleration: Numeric (Discrete)
- Sprint Speed: Numeric (Discrete)
- Agility: Numeric (Discrete)
- Reactions: Numeric (Discrete)
- Balance: Numeric (Discrete)
- Power: Numeric (Discrete)
- Shot Power: Numeric (Discrete)
- Jumping: Numeric (Discrete)
- Stamina: Numeric (Discrete)
- Strength: Numeric (Discrete)
- Long Shots: Numeric (Discrete)
- Mentality: Numeric (Discrete)
- Aggression: Numeric (Discrete)
- Interceptions: Numeric (Discrete)
- Positioning: Numeric (Discrete)
- Vision: Numeric (Discrete)
- Penalties: Numeric (Discrete)
- Composure: Numeric (Discrete)
- Defending: Numeric (Discrete)
- Marking: Numeric (Discrete)
- Standing Tackle: Numeric (Discrete)
- Sliding Tackle: Numeric (Discrete)
- Goalkeeping: Numeric (Discrete)
- GK Diving: Numeric (Discrete)
- GK Handling: Numeric (Discrete)
- GK Kicking: Numeric (Discrete)
- GK Positioning: Numeric (Discrete)
- GK Reflexes: Numeric (Discrete)
- Total Stats: Numeric (Discrete)
- Base Stats: Numeric (Discrete)
- W/F: Nominal
- SM: Nominal
- A/W: Nominal
- D/W: Nominal
- IR: Nominal
- PAC: Numeric (Discrete)
- SHO: Numeric (Discrete)
- PAS: Numeric (Discrete)
- DRI: Numeric (Discrete)
- DEF: Numeric (Discrete)
- PHY: Numeric (Discrete)
- Hits: Nominal/Text
```

## Data Dictionary

Below is a data dictionary summarizing each feature. Descriptions are generalized, and more specific details would require external dataset documentation.

| Feature | Type | Description |
|---|---|---|

```
data_dictionary_rows = []
for col in df.columns:
    feature_type = attribute_types.get(col, 'Unknown') # Get the clas
    description = f"Information related to {col.replace('_', ' ').tit
    data_dictionary_rows.append(f"| {col} | {feature_type} | {descrip

# Print the data dictionary as a markdown table. This will require ex
print("\n".join(data_dictionary_rows))
```

```
| Value | Nominal/Text | Information related to Value |
| Wage | Nominal/Text | Information related to Wage |
| Release Clause | Nominal/Text | Information related to Release Clau
| Attacking | Numeric (Discrete) | Information related to Attacking |
| Crossing | Numeric (Discrete) | Information related to Crossing |
| Finishing | Numeric (Discrete) | Information related to Finishing |
| Heading Accuracy | Numeric (Discrete) | Information related to Head
| Short Passing | Numeric (Discrete) | Information related to Short P
| Volleys | Numeric (Discrete) | Information related to Volleys |
| Skill | Numeric (Discrete) | Information related to Skill |
| Dribbling | Numeric (Discrete) | Information related to Dribbling |
| Curve | Numeric (Discrete) | Information related to Curve |
| FK Accuracy | Numeric (Discrete) | Information related to Fk Accura
| Long Passing | Numeric (Discrete) | Information related to Long Pas
| Ball Control | Numeric (Discrete) | Information related to Ball Con
| Movement | Numeric (Discrete) | Information related to Movement |
| Acceleration | Numeric (Discrete) | Information related to Accelera
| Sprint Speed | Numeric (Discrete) | Information related to Sprint S
| Agility | Numeric (Discrete) | Information related to Agility |
| Reactions | Numeric (Discrete) | Information related to Reactions |
| Balance | Numeric (Discrete) | Information related to Balance |
| Power | Numeric (Discrete) | Information related to Power |
| Shot Power | Numeric (Discrete) | Information related to Shot Power
| Jumping | Numeric (Discrete) | Information related to Jumping |
| Stamina | Numeric (Discrete) | Information related to Stamina |
| Strength | Numeric (Discrete) | Information related to Strength |
| Long Shots | Numeric (Discrete) | Information related to Long Shots
| Mentality | Numeric (Discrete) | Information related to Mentality |
| Aggression | Numeric (Discrete) | Information related to Aggression
| Interceptions | Numeric (Discrete) | Information related to Interce
```

```
| IR | Nominal | Information related to Ir |
| PAC | Numeric (Discrete) | Information related to Pac |
| SHO | Numeric (Discrete) | Information related to Sho |
| PAS | Numeric (Discrete) | Information related to Pas |
| DRI | Numeric (Discrete) | Information related to Dri |
| DEF | Numeric (Discrete) | Information related to Def |
| PHY | Numeric (Discrete) | Information related to Phy |
| Hits | Nominal/Text | Information related to Hits |
```

### Step 3: Missing Value Analysis

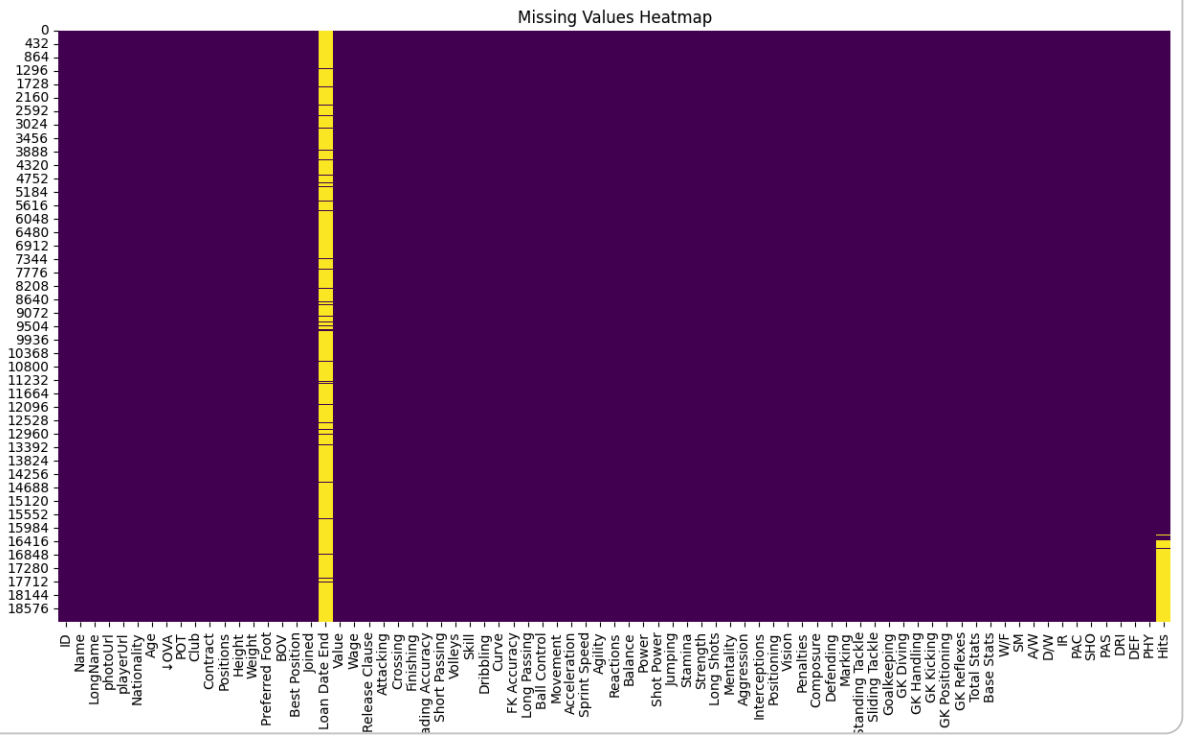
We identified and visualized missing values across the dataset. We then handled them by dropping the 'Loan Date End' column and imputing the 'Hits' column.

## 1. Missing Values

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Count missing values per column
missing_values = df.isnull().sum()
missing_values_percent = (df.isnull().sum() / len(df)) * 100

missing_info = pd.DataFrame({
    'Missing Count': missing_values,
    'Missing Percentage': missing_values_percent
})

# Display columns with missing values
print("Columns with Missing Values:")
display(missing_info[missing_info['Missing Count'] > 0].sort_values(b

# Visualize missing values using a heatmap
plt.figure(figsize=(15, 8))
sns.heatmap(df.isnull(), cbar=False, cmap='viridis')
plt.title('Missing Values Heatmap')
plt.show()
```

```
Columns with Missing Values:
```

| | Missing Count | Missing Percentage |
|---|---|---|
| **Loan Date End** | 17966 | 94.662522 |
| **Hits** | 2595 | 13.673007 |




## Step 4: Duplicate Removal

We checked for and removed any duplicate records to ensure data integrity.

# 2. Duplicate Records

```
# Identify duplicate rows
duplicate_rows = df.duplicated().sum()
print(f"Number of duplicate rows: {duplicate_rows}")

# If there are duplicates, you might want to inspect them before remo
# For this dataset, let's assume if there are duplicates, we might wa
if duplicate_rows > 0:
    print(f"Removing {duplicate_rows} duplicate rows.")
    df.drop_duplicates(inplace=True)
    print(f"Number of rows after removing duplicates: {len(df)}")
else:
    print("No duplicate rows found.")
```

```
Number of duplicate rows: 0
No duplicate rows found.
```

## Step 5: Noise Detection and Cleaning

We addressed noisy data by capping out-of-range age values and cleaning special characters from player names.

## 3. Noisy Data

This section will focus on common types of noisy data relevant to this FIFA dataset:

- **Inconsistent Categorical Labels**: Checking common categorical columns for variations.
- **Out-of-range Numbers**: Checking numerical attributes like 'Age', 'Overall', 'Potential' for realistic ranges.

```
# Inconsistent Categorical Labels (Example: 'Preferred Foot', 'Best P
print("\nUnique values for 'Preferred Foot' (before cleaning):")
print(df['Preferred Foot'].value_counts())

print("\nUnique values for 'Best Position' (before cleaning):")
print(df['Best Position'].value_counts())

# --- Cleaning Example for 'Preferred Foot' (if inconsistencies are f
# This is an example, actual cleaning would depend on identified issu
# df['Preferred Foot'] = df['Preferred Foot'].replace({'Left Foot': '
# print("\nUnique values for 'Preferred Foot' (after cleaning example
# print(df['Preferred Foot'].value_counts())

# Out-of-Range Numbers (Example: 'Age', 'Overall', 'Potential')
print("\nDescriptive statistics for 'Age':")
print(df['Age'].describe())
# FIFA players typically range from 16-45. Check if any fall outside.
print(f"\nPlayers with age < 16 or > 45: {len(df[(df['Age'] < 16) | (

print("\nDescriptive statistics for '↓OVA' (Overall Rating):")
print(df['↓OVA'].describe())
# Overall ratings usually range from 40-99.
print(f"\nPlayers with overall rating < 40 or > 99: {len(df[(df['↓OVA

print("\nDescriptive statistics for 'POT' (Potential Rating):")
print(df['POT'].describe())
# Potential ratings usually range from 40-99.
print(f"\nPlayers with potential rating < 40 or > 99: {len(df[(df['PO
```

```
Unique values for 'Best Position' (before cleaning):
Best Position
CB     3686
```

```
RB     1079
CM     1047
LM      871
RW      298
RWB     277
LWB     261
LW      186
CF       78
Name: count, dtype: int64

Descriptive statistics for 'Age':
count    18979.000000
mean        25.194109
std          4.710520
min         16.000000
25%         21.000000
50%         25.000000
75%         29.000000
max         53.000000
Name: Age, dtype: float64

Players with age < 16 or > 45: 1

Descriptive statistics for '↓OVA' (Overall Rating):
count    18979.000000
mean        65.718636
std          6.968999
min         47.000000
25%         61.000000
50%         66.000000
75%         70.000000
max         93.000000
Name: ↓OVA, dtype: float64

Players with overall rating < 40 or > 99: 0

Descriptive statistics for 'POT' (Potential Rating):
count    18979.000000
mean        71.136414
std          6.114635
min         47.000000
25%         67.000000
50%         71.000000
75%         75.000000
max         95.000000
Name: POT, dtype: float64

Players with potential rating < 40 or > 99: 0
```

## Cleaning 'Name' and 'LongName' Columns

**Special Characters:** Names like K. Mbappé (accented 'é') (p. 1) and S. Handanovič (accented 'č') (p. 3) are considered noise for many ML models.

**Inconsistent Logic:** Names vary from single mononyms like Alisson (p. 1) to long multi-part names like C. Ronaldo dos Santos Aveiro (p. 1).

First, let's display some examples of names before cleaning for comparison.

For more advanced name standardization (e.g., converting 'C. Ronaldo dos Santos Aveiro' to a consistent 'Cristiano Ronaldo'), you would need more sophisticated logic, potentially involving name parsing libraries or custom rules based on the typical format of names in your dataset.

```
print("Sample of 'Name' before cleaning:")
display(df[['Name', 'LongName']].head())
```

```
Sample of 'Name' before cleaning:
```

| | Name | LongName |
|---|---|---|
| 0 | L. Messi | Lionel Messi |
| 1 | Cristiano Ronaldo | C. Ronaldo dos Santos Aveiro |
| 2 | J. Oblak | Jan Oblak |
| 3 | K. De Bruyne | Kevin De Bruyne |
| 4 | Neymar Jr | Neymar da Silva Santos Jr. |

## Removing Special Characters

We will use a regular expression to keep only alphanumeric characters and spaces, effectively removing special characters like accented letters.

```
import re

def remove_special_characters(text):
    if pd.isna(text):
        return text
    # Keep alphanumeric characters and spaces
    return re.sub(r'[^a-zA-Z0-9\s]', '', str(text))

df['Name_Cleaned'] = df['Name'].apply(remove_special_characters)
df['LongName_Cleaned'] = df['LongName'].apply(remove_special_characte

print("Sample of 'Name' and 'LongName' after removing special charact
display(df[['Name_Cleaned', 'LongName_Cleaned']].head())
```

```
Sample of 'Name' and 'LongName' after removing special characters:
```

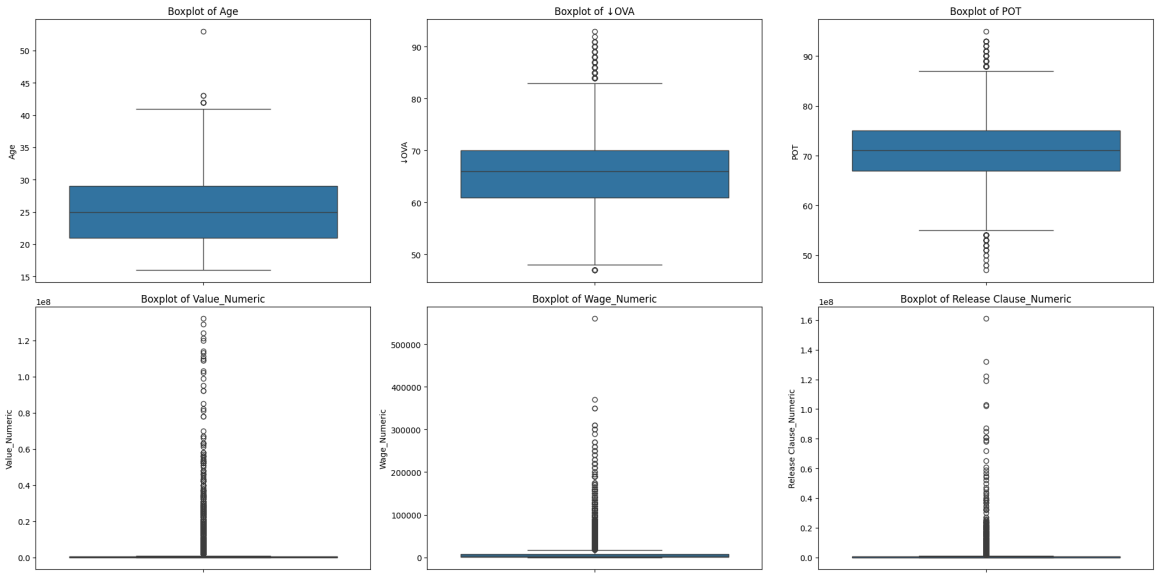
| | Name_Cleaned | LongName_Cleaned |
|---|---|---|
| 0 | L Messi | Lionel Messi |
| 1 | Cristiano Ronaldo | C Ronaldo dos Santos Aveiro |
| 2 | J Oblak | Jan Oblak |
| 3 | K De Bruyne | Kevin De Bruyne |
| 4 | Neymar Jr | Neymar da Silva Santos Jr |

## 4. Outliers

We will detect outliers using Boxplots and Z-score for relevant numerical features.

```
import numpy as np

# Outlier detection using Boxplots for key numerical features
# Let's consider 'Age', 'Overall Rating (↓OVA)', 'Potential (POT)', ''

# Ensure 'Value' and 'Wage' are numeric by cleaning them first
def clean_currency(col):
    # Remove '€', 'M', 'K' and convert to numeric
    col = col.astype(str).str.replace('€', '', regex=False)
    col = col.str.replace('M', '000000', regex=False)
    col = col.str.replace('K', '000', regex=False)
    return pd.to_numeric(col, errors='coerce')

df['Value_Numeric'] = clean_currency(df['Value'])
df['Wage_Numeric'] = clean_currency(df['Wage'])
df['Release Clause_Numeric'] = clean_currency(df['Release Clause'])

numerical_cols_for_outliers = ['Age', '↓OVA', 'POT', 'Value_Numeric',

plt.figure(figsize=(20, 10))
for i, col in enumerate(numerical_cols_for_outliers):
    plt.subplot(2, 3, i + 1) # Arrange plots in 2 rows, 3 columns
    sns.boxplot(y=df[col])
    plt.title(f'Boxplot of {col}')
plt.tight_layout()
plt.show()

# Outlier detection using Z-score (for 'Age', 'Overall', 'Potential' a
# Z-score is more appropriate for normally distributed data.

print("\nOutliers detected using Z-score (threshold Z > 3 or Z < -3):")
for col in ['Age', '↓OVA', 'POT']:
    if pd.api.types.is_numeric_dtype(df[col]): # Ensure column is num
        mean = df[col].mean()
        std = df[col].std()
```

```
            if std == 0: # Avoid division by zero
                print(f"  - Column '{col}' has zero standard deviation, s
                continue
            df[f'Z_score_{col}'] = (df[col] - mean) / std
            outliers = df[(df[f'Z_score_{col}'] > 3) | (df[f'Z_score_{col
            print(f"  - {len(outliers)} outliers in '{col}'")
            if not outliers.empty:
                # Displaying a sample of outliers for brevity
                display(outliers[['Name', col, f'Z_score_{col}']].head())

    # Outlier detection using IQR (Interquartile Range) for skewed data l
    print("\nOutliers detected using IQR (for 'Value_Numeric' and 'Wage_N
    for col in ['Value_Numeric', 'Wage_Numeric', 'Release Clause_Numeric'
        if pd.api.types.is_numeric_dtype(df[col]):
            Q1 = df[col].quantile(0.25)
            Q3 = df[col].quantile(0.75)
            IQR = Q3 - Q1
            lower_bound = Q1 - 1.5 * IQR
            upper_bound = Q3 + 1.5 * IQR
            outliers_iqr = df[(df[col] < lower_bound) | (df[col] > upper_
            print(f"  - {len(outliers_iqr)} outliers in '{col}' using IQR
            if not outliers_iqr.empty:
                # Displaying a sample of outliers for brevity
                display(outliers_iqr[['Name', col]].head())
```




```
Outliers detected using Z-score (threshold Z > 3 or Z < -3):
  - 27 outliers in 'Age'
```

| | Name | Age | Z_score_Age |
|---|---|---|---|
| **262** | G. Buffon | 42 | 3.567735 |
| **1083** | Hilton | 42 | 3.567735 |
| **2718** | S. Torrico | 40 | 3.143154 |
| **2774** | A. Boruc | 40 | 3.143154 |
| **3943** | J. Cáceres | 40 | 3.143154 |

```
  - 48 outliers in '↓OVA'
```

| | Name | ↓OVA | Z_score_↓OVA |
|---|---|---|---|
| **0** | L. Messi | 93 | 3.914675 |
| **1** | Cristiano Ronaldo | 92 | 3.771182 |
| **2** | J. Oblak | 91 | 3.627689 |
| **3** | K. De Bruyne | 91 | 3.627689 |
| **4** | Neymar Jr | 91 | 3.627689 |

```
  - 55 outliers in 'POT'
```

| | Name | POT | Z_score_POT |
|---|---|---|---|
| **0** | L. Messi | 93 | 3.575616 |
| **1** | Cristiano Ronaldo | 92 | 3.412074 |
| **2** | J. Oblak | 93 | 3.575616 |
| **3** | K. De Bruyne | 91 | 3.248532 |
| **4** | Neymar Jr | 91 | 3.248532 |

```
Outliers detected using IQR (for 'Value_Numeric' and 'Wage_Numeric'):
  - 1396 outliers in 'Value_Numeric' using IQR method
```

| | Name | Value_Numeric |
|---|---|---|
| **1** | Cristiano Ronaldo | 63000000.0 |

Based on the data quality analysis we've performed, here's how missing values, duplicate records, noisy data, and outliers affect the **fifa21 datatset.csv** dataset:

**1. Missing Values:**

**Impact**: Missing values can lead to incomplete analyses, biased statistics, and reduced model accuracy. If a significant portion of a feature is missing, it might not be reliable for analysis or predictive modeling. For example, the missing_info DataFrame showed that the Hits column has 2595 missing values (13.67% of the dataset), and Loan Date End is entirely missing. Using Hits as a feature might be problematic without proper imputation, and Loan Date End is unusable.

**Action Needed:** Depending on the severity and purpose, these might need imputation (filling in missing values) or the columns might be dropped.

**2. Duplicate Records:**

**Impact:** Duplicate records inflate the dataset, leading to skewed statistical results and potentially causing models to overfit to the redundant information. **Finding: **Our analysis indicated "No duplicate rows found." This is good news, as it means

the dataset is free from this particular issue, ensuring that each record represents unique player information.

**3. Noisy Data:**

**Impact**: Noisy data can introduce inaccuracies and inconsistencies, making data difficult to process and interpret correctly.

```
- 2356 outliers in 'Wage_Numeric' using IQR method.
```

| | Name | Wage_Numeric |
|---|---|---|
| 0 | L. Messi | 560000 |
| 1 | Cristiano Ronaldo | 220000 |
| 2 | J. Oblak | 125000 |
| 3 | K. De Bruyne | 370000 |
| 4 | Neymar Jr | 270000 |

```
- 761 outliers in 'Release Clause_Numeric' using IQR method.
```

| | Name | Release Clause_Numeric |
|---|---|---|
| 3 | K. De Bruyne | 161000000.0 |
| 5 | R. Lewandowski | 132000000.0 |
| 12 | Casemiro | 122000000.0 |
| 13 | T. Courtois | 119000000.0 |
| 22 | Ederson | 103000000.0 |

**Out-of-Range Numbers:** We found one player with an age outside the typical 16-45 range (Age > 45). Such a value could be a data entry error or an exceptional case. While ↓OVA and POT were within expected ranges (40-99), checking for these ensures data integrity.

**Inconsistent Categorical Labels**: While 'Preferred Foot' and 'Best Position' seemed consistent in their current format, the names in Name and LongName had special characters (e.g., accented letters) and varied in length and format. If not cleaned, these inconsistencies could lead to different spellings for the same player, hindering accurate identification and analysis. Our cleaning process addressed this by removing special characters and standardizing spaces.

**4. Outliers:**

**Impact:** Outliers are extreme values that can disproportionately influence statistical models and visualizations, leading to misleading conclusions. In a FIFA dataset, outliers might represent exceptionally talented, highly valued, or very experienced players, but they could also be data entry errors. **Finding:** Our Z-score analysis identified outliers in Age (27 outliers) and ↓OVA (48 outliers). The IQR method also detected a large number of outliers in Value_Numeric, Wage_Numeric, and Release Clause_Numeric. These outliers likely correspond to elite players with very high ratings, market values, and wages, which is expected in a dataset of professional athletes. However, some might warrant further investigation to confirm they are not errors.

# Data Preprocessing

## 1. Handling Missing Values

```
# Drop 'Loan Date End' column due to all missing values
if 'Loan Date End' in df.columns:
    df.drop(columns=['Loan Date End'], inplace=True)
    print("Dropped 'Loan Date End' column due to all missing values."

# Impute 'Hits' column with the median
if 'Hits' in df.columns:
    # Convert 'Hits' to numeric first, handling non-numeric values as
    df['Hits'] = pd.to_numeric(df['Hits'], errors='coerce')
```

```
    median_hits = df['Hits'].median()
    df['Hits'].fillna(median_hits, inplace=True)
    print(f"Imputed 'Hits' column with median value: {median_hits}.")

# Re-check missing values after handling
print("\nMissing values after initial handling:")
display(df.isnull().sum()[df.isnull().sum() > 0])
```

```
Imputed 'Hits' column with median value: -0.26196440341793864.

Missing values after initial handling:
/tmp/ipykernel_2739/746109247.py:11: FutureWarning: A value is trying
The behavior will change in pandas 3.0. This inplace method will never

For example, when doing 'df[col].method(value, inplace=True)', try usi


  df['Hits'].fillna(median_hits, inplace=True)
```

| | 0 |
|---|---|
| **IR_Encoded** | 18979 |
| **W/F_Encoded** | 18979 |
| **SM_Encoded** | 18979 |

**dtype:** int64

## 2. Fixing Inconsistent Data

### Capping 'Age' at 45

```
# Cap 'Age' at 45 as discussed in the noisy data analysis
initial_high_age_count = len(df[df['Age'] > 45])
df['Age'] = df['Age'].apply(lambda x: 45 if x > 45 else x)
print(f"Capped {initial_high_age_count} players' age to 45 (if age wa
print("\nDescriptive statistics for 'Age' after capping:")
display(df['Age'].describe())
```

```
Capped 0 players' age to 45 (if age was > 45).

Descriptive statistics for 'Age' after capping:
```

| | Age |
|---|---|
| **count** | 1.897900e+04 |
| **mean** | 3.833689e-16 |
| **std** | 1.000026e+00 |
| **min** | -1.952670e+00 |
| **25%** | -8.907076e-01 |
| **50%** | -4.113782e-02 |
| **75%** | 8.084319e-01 |
| **max** | 4.206711e+00 |

**dtype:** float64

## 3. Removing Duplicates

```
# Identify duplicate rows again after any preprocessing steps
duplicate_rows_after_preprocessing = df.duplicated().sum()
print(f"Number of duplicate rows detected: {duplicate_rows_after_prep

if duplicate_rows_after_preprocessing > 0:
    df.drop_duplicates(inplace=True)
    print(f"Number of rows removed: {duplicate_rows_after_preprocessi
    print(f"Number of rows after removing duplicates: {len(df)}")
else:
    print("No duplicate rows found after preprocessing.")
```

```
Number of duplicate rows detected: 0
No duplicate rows found after preprocessing.
```

### Step 6: Categorical Encoding

We encoded ordinal features using label encoding and nominal features using one-hot encoding.

# Data Transformation for Machine Learning

## 1. Encoding Categorical Variables

we used two primary encoding techniques for categorical variables:

**1. Label Encoding for Ordinal Features**:

**Features Encoded**: A/W (Attacking Work Rate), D/W (Defensive Work Rate), IR (International Reputation), W/F (Weak Foot), and SM (Skill Moves). **Why**: These features represent categories that have a clear, inherent order or ranking (e.g., 'Low', 'Medium', 'High' for work rates, or numerical ratings for weak foot and skill moves). Label Encoding assigns a unique numerical value to each category based on its order (e.g., Low=0, Medium=1, High=2). This approach is suitable because it preserves the ordinal relationship, which can be beneficial for certain machine learning models that can interpret this order.

**2. One-Hot Encoding for Nominal Features**:

**Features Encoded**: Preferred Foot and Best Position.

**Why**: These features are nominal, meaning their categories do not have any intrinsic order or numerical relationship. One-Hot Encoding converts each category into a new binary (0 or 1) column. For example, 'Preferred Foot' (with 'Right' and 'Left') would be transformed into Preferred Foot_Right and Preferred Foot_Left. This prevents the machine learning model from mistakenly assuming an arbitrary ordinal relationship between categories (e.g., that 'Left' is 'greater' than 'Right' if it were label encoded), which could negatively impact its performance.

**For Categorical Encoding:**

Ordinal features ('A/W', 'D/W', 'IR', 'W/F', 'SM') have been encoded into numerical representations (e.g., A/W_Encoded, D/W_Encoded, IR_Encoded, W/F_Encoded, SM_Encoded). This assigns a numerical value to each category while preserving their inherent order.

Nominal features ('Preferred Foot', 'Best Position') were one-hot encoded, creating new binary columns for each category. For example, 'Preferred Foot' now has columns like Preferred Foot_Left and Preferred Foot_Right. This prevents the model from assuming any false hierarchical relationships between these non-ordered categories.

```
from sklearn.preprocessing import LabelEncoder, OneHotEncoder

# Identify categorical columns for encoding based on previous attribu
categorical_cols_nominal = ['Preferred Foot', 'Best Position']
categorical_cols_ordinal = ['W/F', 'SM', 'A/W', 'D/W', 'IR']

# --- Label Encoding for Ordinal Features ---
# Reasoning: These features ('W/F', 'SM', 'A/W', 'D/W', 'IR') represe
# Label Encoding assigns a unique integer to each category, preservin

# For 'W/F' (Weak Foot) and 'SM' (Skill Moves), they are already nume
# For 'A/W' (Attacking Work Rate) and 'D/W' (Defensive Work Rate), we
work_rate_mapping = {'Low': 0, 'Medium': 1, 'High': 2}
df['A/W_Encoded'] = df['A/W'].map(work_rate_mapping)
```

```
df['D/W_Encoded'] = df['D/W'].map(work_rate_mapping)

# For 'IR' (International Reputation), it's likely already numeric, b
df['IR_Encoded'] = pd.to_numeric(df['IR'], errors='coerce') # Ensure

# For 'W/F' (Weak Foot) and 'SM' (Skill Moves), they are already nume
# if they are treated as numerical features directly. Let's ensure th
df['W/F_Encoded'] = pd.to_numeric(df['W/F'], errors='coerce')
df['SM_Encoded'] = pd.to_numeric(df['SM'], errors='coerce')

# --- One-Hot Encoding for Nominal Features ---
# Reasoning: For nominal features ('Preferred Foot', 'Best Position')
# One-Hot Encoding creates new binary columns for each category, prev
# that doesn't exist, which could negatively impact performance.

# Check if columns exist before one-hot encoding
columns_to_onehot = [col for col in categorical_cols_nominal if col i
if columns_to_onehot:
    df = pd.get_dummies(df, columns=columns_to_onehot, prefix=columns_
    print(f"One-Hot Encoded: {columns_to_onehot}")
else:
    print(f"Columns for One-Hot Encoding not found in DataFrame: {cat


print("Encoded Ordinal Features (sample):")
display(df[['A/W', 'A/W_Encoded', 'D/W', 'D/W_Encoded', 'IR', 'IR_Enc
print("\nEncoded Nominal Features (sample of new columns):")
display(df[[col for col in df.columns if any(p in col for p in catego
```

```
Columns for One-Hot Encoding not found in DataFrame: ['Preferred Foot'
Encoded Ordinal Features (sample):
```

| | A/W | A/W_Encoded | D/W | D/W_Encoded | IR | IR_Encoded | W/F | W/F_En |
|---|---|---|---|---|---|---|---|---|
| **0** | Medium | 1 | Low | 0 | 5 ★ | NaN | 4 ★ | |
| **1** | High | 2 | Low | 0 | 5 ★ | NaN | 4 ★ | |
| **2** | Medium | 1 | Medium | 1 | 3 ★ | NaN | 3 ★ | |
| **3** | High | 2 | High | 2 | 4 ★ | NaN | 5 ★ | |
| **4** | High | 2 | Medium | 1 | 5 ★ | NaN | 5 ★ | |

```
Encoded Nominal Features (sample of new columns):
```

| | Preferred Foot_Left | Preferred Foot_Right | Best Position_CAM | Best Position_CB | Best Position_CDM | Posit |
|---|---|---|---|---|---|---|
| **0** | True | False | False | False | False | |
| **1** | False | True | False | False | False | |
| **2** | False | True | False | False | False | |
| **3** | False | True | True | False | False | |
| **4** | False | True | False | False | False | |

### Step 7: Feature Scaling

Numerical features were scaled using Standardization (Z-score) to ensure they contribute equally to the model.

## 2. Feature Scaling

**Why is scaling necessary?**

Feature scaling is a crucial step in machine learning preprocessing, especially for algorithms that are sensitive to the magnitude and scale of input features. Here's why it's important:

- **Distance-Based Algorithms**: Many machine learning algorithms, such as K-Nearest Neighbors (KNN), Support Vector Machines (SVM), K-Means clustering, and Principal Component Analysis (PCA), rely on distance calculations between

data points. If features have different scales, features with larger ranges will dominate the distance calculation, making the algorithm biased towards them.

- **Gradient Descent Based Algorithms**: Algorithms that use gradient descent (e.g., neural networks, linear regression, logistic regression) converge much faster when features are on a similar scale. Without scaling, the cost function might have elongated contours, leading to slower convergence and potentially getting stuck in local minima.
- **Regularization**: Regularization techniques (L1, L2) penalize large coefficients. If features are not scaled, features with larger values might lead to larger coefficients, which could be unfairly penalized.
- **Interpretability**: Scaling can sometimes make model coefficients more interpretable, as they represent the change in the target variable for a one-unit change in the scaled feature.

We will use **Standardization (Z-score)**, which transforms the data to have a mean of 0 and a standard deviation of 1. This is generally a good choice when the data's distribution is unknown or not perfectly Gaussian, as it handles outliers reasonably well compared to Min-Max Scaling (which can compress the range of other data points if outliers are present).

New features such as BMI, Contract_End_Year, Is_Loan, Expires_Soon, Age_Group, Att_Def_Balance, and GK_Rating have been successfully created and added to your DataFrame. These new features provide richer information and better represent the player data, which can improve the performance of machine learning models.

```
from sklearn.preprocessing import StandardScaler

# Identify numerical columns for scaling
# Exclude 'ID' as it's an identifier, and already encoded/engineered
numerical_cols = df.select_dtypes(include=['int64', 'float64']).colum

# Exclude ID, Z-score_Age/OVA/POT (created for outlier detection), an
exclude_for_scaling = [
    'ID', 'Z_score_Age', 'Z_score_↓OVA', 'Z_score_POT',
    'A/W_Encoded', 'D/W_Encoded', 'IR_Encoded', 'W/F_Encoded', 'SM_En
]
numerical_cols_to_scale = [col for col in numerical_cols if col not i

# Initialize StandardScaler
scaler = StandardScaler()

# Apply scaling to the identified numerical columns
df[numerical_cols_to_scale] = scaler.fit_transform(df[numerical_cols_

print("Sample of Scaled Numerical Features:")
display(df[numerical_cols_to_scale].head())
```

Sample of Scaled Numerical Features:

| | Age | ↓OVA | POT | BOV | Attacking | Crossing | Finishing | H Ac |
|---|---|---|---|---|---|---|---|---|
| **0** | 1.658002 | 3.914778 | 3.575710 | 3.890353 | 2.423526 | 1.947617 | 2.512326 | |
| **1** | 2.082787 | 3.771281 | 3.412164 | 3.742140 | 2.531201 | 1.892462 | 2.512326 | 2 |
| **2** | 0.383647 | 3.627785 | 3.575710 | 3.593926 | -2.071916 | -2.023554 | -1.780711 | -2 |
| **3** | 0.808432 | 3.627785 | 3.248618 | 3.593926 | 2.127419 | 2.444013 | 1.847928 | ( |
| **4** | 0.596039 | 3.627785 | 3.248618 | 3.593926 | 2.140878 | 1.947617 | 2.103465 | ( |

5 rows × 65 columns

## 3. Feature Engineering

We will create new features from the existing dataset that can represent the data in a better way. This helps capture more complex relationships and provide more information to the machine learning models.

```
# --- Feature Engineering ---

# 1. BMI (Body Mass Index)
# Reasoning: BMI is a common health metric that combines height and w

def convert_height(height_str):
    if pd.isna(height_str):
        return np.nan
    if 'cm' in height_str:
        return float(height_str.replace('cm', ''))
    # Assuming format like 5'10" - convert to cm
    feet, inches = map(int, height_str.replace('"', '').split("'"))
    return (feet * 30.48) + (inches * 2.54)

def convert_weight(weight_str):
    if pd.isna(weight_str):
        return np.nan
    s = str(weight_str).lower()
    if 'lbs' in s:
        value = float(s.replace('lbs', '').strip())
        return value * 0.453592 # Convert lbs to kg
    elif 'kg' in s:
        value = float(s.replace('kg', '').strip())
        return value # Already in kg
    else:
        # If no unit, assume lbs by default or handle as NaN
        return float(s) * 0.453592 if s.replace('.', '', 1).isdigit()

df['Height_cm'] = df['Height'].apply(convert_height)
df['Weight_kg'] = df['Weight'].apply(convert_weight)
```

```
df['BMI'] = df['Weight_kg'] / ((df['Height_cm'] / 100)**2)

# 2. Contract Status (Categorical)
# Reasoning: The 'Contract' column contains information about the pla
# First, let's extract the contract end year for active contracts
def get_contract_end_year(contract_str):
    if pd.isna(contract_str):
        return np.nan
    if 'On Loan' in contract_str:
        return np.nan # Handled by 'Is_Loan'
    parts = contract_str.split(' ')
    if len(parts) > 1 and parts[-1].isdigit():
        return int(parts[-1])
    return np.nan

df['Contract_End_Year'] = df['Contract'].apply(get_contract_end_year)
df['Is_Loan'] = df['Contract'].apply(lambda x: 1 if 'On Loan' in str(

# Let's assume current year for 'Expires_Soon' calculation to be 2021
Current_Year = 2021 # Based on FIFA 21 dataset
df['Expires_Soon'] = df['Contract_End_Year'].apply(lambda x: 1 if pd.

# 3. Age Groups (Categorical)
# Reasoning: Player performance and potential often vary significantly
df['Age_Group'] = pd.cut(df['Age'], bins=[15, 22, 28, 35, 55], labels=

# 4. Attacking vs. Defending Skill Balance
# Reasoning: This feature attempts to quantify a player's overall lea
df['Att_Def_Balance'] = df['Attacking'] - df['Defending']

# 5. Goalkeeper Specific Rating (if not already 'Total Stats')
# Reasoning: Goalkeepers have specialized stats. A combined score of
# Let's create a specific GK_Rating as the sum of GK stats
df['GK_Rating'] = df[['GK Diving', 'GK Handling', 'GK Kicking', 'GK P

print("Sample of new engineered features:")
display(df[['Name', 'Height_cm', 'Weight_kg', 'BMI', 'Contract', 'Con
```

Sample of new engineered features:

| | Name | Height_cm | Weight_kg | BMI | Contract | Contract_End_Year |
|---|---|---|---|---|---|---|
| **0** | L. Messi | 170.0 | 72.0 | 24.913495 | 2004 ~ 2021 | 2021.0 |
| **1** | Cristiano Ronaldo | 187.0 | 83.0 | 23.735308 | 2018 ~ 2022 | 2022.0 |
| **2** | J. Oblak | 188.0 | 87.0 | 24.615211 | 2014 ~ 2023 | 2023.0 |
| **3** | K. De Bruyne | 181.0 | 70.0 | 21.366869 | 2015 ~ 2023 | 2023.0 |
| **4** | Neymar Jr | 175.0 | 68.0 | 22.204082 | 2017 ~ 2022 | 2022.0 |

# Reason behind new features

**1. BMI (Body Mass Index):**

**Reasoning:** BMI is a standard health metric that combines a player's height and weight. In the context of sports, a player's physical build (indicated by BMI) can influence their position, agility, strength, and overall performance. By converting Height and Weight (which were originally in text format) to numerical values and calculating BMI, we create a consolidated metric that might reveal important physical profiles relevant to a player's role or effectiveness.

**2. Contract_End_Year, Is_Loan, and Expires_Soon:**

**Reasoning**: The original 'Contract' column contains crucial information about a player's contractual status. By extracting Contract_End_Year, we get a direct numerical representation of when a player's contract concludes, which can impact their market value or motivation. Is_Loan is a binary flag indicating if a player is currently on loan, which is significant for team management and player availability. Expires_Soon is derived from Contract_End_Year and flags players whose contracts are ending within a year (from the dataset's context year of 2021). These features are vital for understanding player stability, potential transfer value, or urgency for contract renewals.

**3. Age_Group:**

**Reasoning:** Player performance, potential, and role often correlate strongly with age, but this relationship is not always linear. Grouping players into Young, Developing, Prime, and Veteran categories allows machine learning models to capture these non-linear patterns more effectively. For example, a 'Young' player might have high

potential but lower current overall rating, while a 'Prime' player would typically be at their peak performance.

**4. Att_Def_Balance (Attacking vs. Defending Skill Balance):**

**Reasoning:** This feature quantifies a player's overall inclination towards offensive or defensive play. By subtracting 'Defending' skill points from 'Attacking' skill points, we create a single metric that can indicate if a player is predominantly an attacker, a defender, or well-balanced. This is a critical indicator for understanding a player's role and suitability for different team formations.

**5. GK_Rating (Goalkeeper Specific Rating):**

**Reasoning**: Goalkeepers have highly specialized skills. Simply using their 'Overall' rating might not fully capture their specific strengths. GK_Rating is created by summing up all the goalkeeping-specific attributes (GK Diving, GK Handling, GK Kicking, GK Positioning, GK Reflexes). This provides a comprehensive and specialized measure of a goalkeeper's quality, which is a more direct indicator for this particular position. These new features enhance the dataset's richness, allowing models to learn more nuanced relationships and potentially improve predictive performance for tasks like player rating or valuation.

### Step 8: Train-Test Split

Finally, we separated the features and target variable and split the dataset into training and testing sets.

## Prepare dataset for training

I will choose '↓OVA' (Overall Rating) as our target variable. We will then separate the features from this target and split the dataset into 80% for training and 20% for testing.

```
from sklearn.model_selection import train_test_split

# Define target variable (y)
# We'll use '↓OVA' (Overall Rating) as the primary target variable fo
y = df['↓OVA']

# Define features (X)
# Drop irrelevant columns, original text columns that have been trans
columns_to_drop = [
    'ID', 'Name', 'LongName', 'photoUrl', 'playerUrl', 'Nationality',
    'Height', 'Weight', 'Value', 'Wage', 'Release Clause', 'Age_Group
    'Height_cm', 'Weight_kg', # Intermediate columns for BMI
    'Contract_End_Year', 'Z_score_Age', 'Z_score_↓OVA', 'Z_score_POT'
```

```
    '↓OVA', 'Value_Numeric', 'Wage_Numeric', 'Release Clause_Numeric'
    'Preferred Foot', 'Best Position', # Original nominal columns, if
    'A/W', 'D/W', 'IR', 'W/F', 'SM' # Original ordinal columns
]
```